# CALC — Calculadora

## Plano de Gestão e Configuração de Software (PGCS) — Enfoque DevOps (CI/CD)

**Versão 1.1** — atualizada em 28/10/2026

| Campo | Descrição |
|---|---|
| Projeto | CALC — Calculadora |
| Gerente de Projeto | Elisa Maria Gomes (@elisamgomes) |
| Gerente/Responsável de GCS/DevOps | Elisa Maria Gomes |
| Revisor par (acesso de escrita) | @otavioroecker |
| Contexto | Atividade Avaliativa de Gerenciamento de Configuração de Software |
| Repositório principal | https://github.com/elisamgomes/calculadora (público) |

### Histórico de Revisões

| Data | Versão | Descrição | Autor |
|---|---|---|---|
| 23/09/2026 | 1.0 | Elaboração do documento com enfoque em CI, a partir do template institucional e do repositório. | Elisa Maria Gomes |
| 28/10/2026 | 1.1 | Atualização para o estado atual: CD com Environments staging e production, proteção da branch main, revisão por par, template de Pull Request, Conventional Commits e evidências. | Elisa Maria Gomes |

## 1. Introdução

Este PGCS descreve as atividades de Gestão de Configuração de Software (GCS) do projeto Calculadora, com ênfase em práticas DevOps: Integração Contínua (CI) e Entrega Contínua (CD). As atividades envolvem identificar os itens de configuração, manter sua

integridade e controlar as mudanças de forma sistemática, com o mínimo de burocracia e o máximo de automação.

O plano é ele próprio um item de configuração: está publicado no repositório e só pode ser alterado por Pull Request revisado.

## 1.1 Finalidade

Estabelecer o padrão seguido no projeto para garantir rastreabilidade, versionamento e controle de mudanças, apoiado por um pipeline automatizado de build, lint, testes e entrega em ambientes, e por uma política de revisão por pares.

## 1.2 Escopo

O plano cobre o projeto Calculadora (aplicação de linha de comando em Python): o código-fonte ( `calculadora.py` ), os testes automatizados ( `tests/test_calculadora.py` ), os pipelines ( `.github/workflows/ci.yml` e `cd.yml` ), o template de Pull Request e a documentação ( `README.md` , este PGCS e as evidências em `docs/` ).

Como a aplicação é uma CLI sem servidor, o deploy em staging e production é **simulado** no pipeline (registra a publicação do pacote gerado), com o objetivo de exercitar e comprovar o fluxo de promoção entre ambientes, a proteção do ambiente de produção e a aprovação manual. Infraestrutura como Código segue como proposta (seção 7).

## 1.3 Definições, Acrônimos e Abreviações

| Termo | Significado |
|---|---|
| GCS | Gestão de Configuração de Software |
| CI | Integração Contínua (Continuous Integration) |
| CD | Entrega/Implantação Contínua (Continuous Delivery / Deployment) |
| IaC | Infraestrutura como Código |
| PR | Pull Request |
| Lint | Análise estática de estilo (ferramenta: flake8) |
| Ruleset | Regra de proteção de branch do GitHub |
| Environment | Ambiente do GitHub (staging, production) com regras de proteção próprias |

| Termo | Significado |
|---|---|
| Baseline | Conjunto de itens de configuração em estado estável e comprovado |
| SemVer | Versionamento Semântico (MAJOR.MINOR.PATCH) |

### 1.4 Referências

- Documento base: Processos de Desenvolvimento de Software e a Integração com GCS.
- Template do Plano de Gerenciamento de Configuração (RUP 7.0, IBM), adaptado.
- Repositório do projeto: https://github.com/elisamgomes/calculadora.
- Repositório de referência da disciplina: https://github.com/lzcamargo/ongconecta.
- Documentação oficial do GitHub Actions, rulesets e Environments.

### 1.5 Visão Geral

A GCS é tratada como parte constante do fluxo de trabalho. Nenhuma mudança entra na `main` sem passar por Pull Request, pelo CI verde e pela aprovação do revisor par. Após o merge, o CD gera o pacote, publica em staging automaticamente e aguarda a aprovação da responsável para production. Cada etapa deixa evidências no histórico do Git e nas execuções do GitHub Actions.

## 2. Cultura DevOps e Papéis na Gerência de Configuração

| Papel | Responsabilidades | Exercido por |
|---|---|---|
| Gerente de Projeto / Product Owner | Prioriza melhorias, acompanha a qualidade das entregas e aprova o deploy em production. | Elisa Maria Gomes |
| Engenheiro DevOps / Release Engineer | Projeta e mantém os pipelines ( `ci.yml` , `cd.yml` ), os Environments, o ruleset e a estratégia de branches. | Elisa Maria Gomes |
| Desenvolvedora | Escreve o código-fonte e os testes e abre os Pull Requests. | Elisa Maria Gomes |
| Revisor par (Reviewer) | Revisa e aprova os Pull Requests (1 aprovação obrigatória). Tem acesso de escrita ao repositório como Collaborator. | @otavioroecker |

| Papel | Responsabilidades | Exercido por |
|---|---|---|
| QA / Automação de Testes | Mantém a suíte pytest, executada a cada PR e a cada push na main. | Elisa Maria Gomes |
| Segurança (DevSecOps) | Não aplicável: não há segredos nem análise automatizada de dependências (SCA). | N/A |
| Comitê de Controle de Mudanças (CCM) | Substituído pelo par de revisão e pelo gate de aprovação de production (seção 8.2). | Par de revisão + Elisa |

Por se tratar de repositório em conta pessoal do GitHub, todo Collaborator possui acesso de escrita; o GitHub não oferece níveis intermediários de permissão nesse tipo de repositório.

# 3. Ferramentas, Ambientes e Infraestrutura

## 3.1 Ferramentas do Pipeline

| Categoria | Ferramenta adotada | Finalidade |
|---|---|---|
| Controle de versão | Git + GitHub | Versionamento do código, testes, pipelines e documentação. |
| Orquestração de CI/CD | GitHub Actions | Execução automática de CI em PRs e de CD na main. |
| Ambiente de execução | actions/setup-python@v5 (Python 3.12), runner ubuntu-latest | Runner recriado a cada execução, garantindo reprodutibilidade. |
| Lint | flake8 ( `--max-line-length=100` ) | Análise estática de estilo. |
| Testes | pytest ( `python -m pytest` ) | Testes unitários das quatro operações e da divisão por zero. |
| Dependências | requirements.txt (pytest, flake8) | Declaração e instalação reprodutível. |

| Categoria | Ferramenta adotada | Finalidade |
|---|---|---|
| Proteção de branch | GitHub Rulesets (ruleset `proteger-main`) | Impede push direto na main e exige PR, aprovação e CI verde. |
| Ambientes | GitHub Environments (`staging`, `production`) | Promoção controlada entre ambientes; production com revisor obrigatório. |
| Artefatos | actions/upload-artifact e download-artifact | Pacote `calculadora-dist` gerado no build e reutilizado nos deploys. |
| Padronização de PR | `.github/pull_request_template.md` | Formulário Contexto / Decisão / Como testar. |
| Rastreamento de tarefas | GitHub Issues | Registro de melhorias e correções. |
| Contêineres / IaC / Segredos / Monitoramento | Não aplicável | Projeto CLI sem infraestrutura provisionada. |

## 3.2 Ambientes e Infraestrutura

| Ambiente | Promoção | Proteção |
|---|---|---|
| Revisão (PR) | Execução do CI a cada Pull Request | Check `build-and-test` obrigatório |
| `staging` | Automática, após build e testes verdes na main | Sem aprovação manual |
| `production` | Após staging | **Revisor obrigatório: Elisa Maria Gomes (@elisamgomes)** |

Todos os ambientes de execução são runners efêmeros do GitHub Actions. Os deploys são simulados (seção 1.2).

## 3.3 Estratégia de Branching e Versionamento

- `main`: branch estável e **protegida pelo ruleset `proteger-main`**. Não aceita push direto, nem force push, nem exclusão.
- `feature/<n>-<descrição>`: novas funcionalidades.

- `fix/<n>-<descrição>`: correções.
- `docs/<descrição>`: alterações de documentação, inclusive deste plano e das evidências.
- As branches partem da `main` atualizada e são removidas após o merge. Não há branches de release ou hotfix, dado o porte do projeto.

**Padrão de commits:** Conventional Commits (`feat`, `fix`, `docs`, `test`, `chore`, `ci`). O título do Pull Request segue o padrão, pois, com o **squash merge**, ele se torna o commit na `main`.

**Versionamento:** Versionamento Semântico (`feat` incrementa MINOR, `fix` incrementa PATCH, mudança incompatível incrementa MAJOR). A geração automática de tags continua como proposta (seção 9).

# 4. Identificação da Configuração

## 4.1 Itens de Configuração

Itens versionados em https://github.com/elisamgomes/calculadora:

- Código-fonte da aplicação (`calculadora.py`)
- Testes automatizados (`tests/test_calculadora.py`)
- Pipeline de CI como código (`.github/workflows/ci.yml`)
- Pipeline de CD como código (`.github/workflows/cd.yml`)
- Template de Pull Request (`.github/pull_request_template.md`)
- Arquivo de dependências (`requirements.txt`)
- Arquivo de exclusão de versionamento (`.gitignore`)
- Documentação técnica (`README.md`, este PGCS e o plano `docs/plano-gcs.md`)
- Evidências de conformidade (`docs/evidencias/`)
- Configurações do repositório no GitHub (ruleset, Environments e colaboradores), comprovadas pelas evidências, pois não são versionadas em arquivo.

Não são itens de configuração: Dockerfiles, manifests de contêineres, scripts de IaC e migrações de banco de dados (N/A).

## 4.2 Convenção de Nomenclatura

| Item | Convenção | Exemplo |
|---|---|---|
| Branch de feature | `feature/<n>-<descrição-curta>` | `feature/12-operacao-potenciacao` |

| Item | Convenção | Exemplo |
|---|---|---|
| Branch de documentação | `docs/<descrição>` | `docs/evidencias` |
| Commit / título de PR | `<tipo>(<escopo>): <descrição>` | `feat(calculadora): adiciona potenciação` |
| Tag de release (proposta) | `vX.Y.Z` | `v1.0.0` |
| Evidências | nome descritivo em `docs/evidencias/` | `push-recusado.png` |

### 4.3 Estrutura do Repositório

```
calculadora/
├── .github/
│   ├── workflows/
│   │   ├── ci.yml                  — pipeline de CI (Pull Requests)
│   │   └── cd.yml                  — pipeline de CD (push na main)
│   └── pull_request_template.md    — modelo de descrição de PR
├── docs/
│   ├── plano-gcs.md                — plano de GCS publicado no repositório
│   └── evidencias/                 — prints que comprovam as configurações
├── tests/
│   └── test_calculadora.py         — testes automatizados (pytest)
├── calculadora.py                  — código-fonte da aplicação
├── requirements.txt                — dependências (pytest, flake8)
├── .gitignore                      — arquivos ignorados pelo Git
└── README.md                       — documentação do projeto
```

## 5. Integração Contínua (CI)

Todo Pull Request para a `main` dispara o CI ( `.github/workflows/ci.yml` ). O job `build-and-test` é um **check obrigatório** no ruleset: sem ele verde, o merge fica bloqueado.

### 5.1 Etapas do Pipeline de CI

1. Checkout do código (actions/checkout@v4).
2. Instalação do Python 3.12 (actions/setup-python@v5).
3. Instalação das dependências ( `pip install -r requirements.txt` ).
4. Lint ( `flake8 . --max-line-length=100` ).

5. Testes automatizados ( `python -m pytest` ).

### 5.2 Política de Pull Request e Revisão de Código

- Nenhuma alteração entra na `main` por push direto: toda mudança passa por Pull Request.
- O título do PR segue Conventional Commits e a descrição usa o template (Contexto / Decisão / Como testar).
- São exigidos **1 aprovação** do revisor par e o **CI verde**. Quem abre o PR não aprova o próprio PR.
- O merge é feito por **squash merge**, mantendo o histórico da `main` linear e rastreável.
- Prazo de resposta a Pull Requests: até 3 dias úteis.

### 5.3 Critérios de Qualidade (Quality Gate)

- 100% dos testes automatizados passando.
- Zero erros de lint no flake8.
- Check `build-and-test` com sucesso.
- 1 aprovação de revisão.

## 6. Entrega e Implantação Contínua (CD)

O CD está implementado em `.github/workflows/cd.yml` , com gatilho em **push na `main`** (ou seja, após o merge de um PR).

### 6.1 Etapas do Pipeline de CD

| Job | Descrição | Ambiente / proteção |
|---|---|---|
| `build` | Instala dependências, roda `python -m pytest` e gera o pacote `calculadora-dist` (código, dependências e README) como artefato. | — |
| `staging` | Baixa o artefato e executa o deploy (simulado) em staging. | Environment `staging` , automático |
| `production` | Baixa o mesmo artefato e executa o deploy (simulado) em produção. | Environment `production` , **aguarda aprovação de Elisa Maria Gomes** |

O mesmo artefato percorre os dois ambientes, garantindo que o que foi testado é o que é promovido. A aprovação é feita na aba **Actions**, em *Review deployments*.

Evoluções propostas, ainda não implementadas: testes de aceitação em staging, smoke tests pós-deploy, geração automática de tag e Release.

### 6.2 Estratégias de Implantação (referência)

| Estratégia | Descrição |
|---|---|
| Rolling update | Substituição gradual das instâncias antigas pelas novas. |
| Blue-Green | Duas versões coexistem; o tráfego é comutado de uma vez, com rollback imediato. |
| Canary Release | A nova versão recebe uma fração do tráfego, ampliada gradualmente. |

O projeto usa promoção sequencial staging → production com gate manual, adequada ao baixo risco e ao deploy simulado.

### 6.3 Rollback e Recuperação

Em caso de problema, reverte-se o commit ou o PR responsável com `git revert`, por meio de um novo Pull Request que segue o fluxo normal (CI, aprovação e CD). Alternativamente, o pacote de uma execução anterior bem-sucedida pode ser reimplantado pela aba Actions (*Re-run all jobs*), e, quando houver tags, republicando-se a versão anterior.

## 7. Infraestrutura como Código (IaC)

Não aplicável: não há infraestrutura provisionada. Os pipelines ( `ci.yml` e `cd.yml` ) são versionados como código e sujeitos ao mesmo fluxo de Pull Request. Caso o projeto evolua para uma aplicação implantável, a infraestrutura deverá ser descrita como código (por exemplo, Terraform ou Dockerfile) e submetida a esse mesmo fluxo.

## 8. Controle de Configuração e Mudança

### 8.1 Processo de Solicitação de Mudança

1. Registro da melhoria ou correção em uma Issue.
2. Criação de branch a partir da `main` atualizada ( `feature/` , `fix/` ou `docs/` ).

3. Alterações em commits pequenos, no padrão Conventional Commits.
4. Abertura de Pull Request preenchendo o template.
5. Execução automática do CI; o check `build-and-test` precisa estar verde.
6. Revisão e aprovação do revisor par.
7. Squash merge na `main`.
8. CD: build, staging automático e aprovação manual para production.

Esse fluxo é o registro rastreável da mudança no histórico do Git, nos Pull Requests e nas execuções do GitHub Actions.

### 8.2 Controle de Mudança (substituto do CCM)

Não há um comitê formal. O controle é exercido por dois gates: (1) a aprovação do revisor par no Pull Request, que autoriza a mudança a entrar na `main`; e (2) a aprovação da responsável pelo projeto no Environment `production`, que autoriza a promoção para produção.

### 8.3 Proteção da Branch main

Ruleset `proteger-main`, com status **Active**, alvo na branch `main` e lista de bypass vazia, de modo que a regra vale também para a dona do repositório:

- exigir Pull Request antes do merge, com 1 aprovação;
- exigir o check de status `build-and-test`;
- bloquear force push;
- restringir a exclusão da branch.

A recusa do commit direto na `main` está comprovada nas evidências (seção 15).

## 9. Baselines e Releases

### 9.1 Geração de Baseline

Cada merge na `main` com CI e CD verdes é uma baseline implícita, identificada pelo commit e pela execução do pipeline correspondente. A marcação formal por tag de versão (SemVer) permanece como melhoria proposta e ainda não foi implementada, assim como a criação automática de Release no GitHub.

### 9.2 Notas de Versão (Release Notes)

Com Conventional Commits e squash merge, o histórico da `main` já é legível por tipo de mudança, o que permite gerar notas de versão semiautomáticas quando as tags forem adotadas.

## 10. Estimativa do Status de Configuração

### 10.1 Monitoramento e Observabilidade

Não há ambiente de produção real a monitorar. O status é acompanhado pela aba Actions, que mostra o resultado de cada execução de CI e CD, e pela aba Environments, que registra os deploys e as aprovações.

### 10.2 Métricas DORA (adaptadas ao contexto)

| Métrica | Aplicabilidade |
|---|---|
| Lead Time for Changes | Aplicável: tempo entre o commit e a conclusão do pipeline até production. |
| Deployment Frequency | Parcialmente aplicável: conta os deploys (simulados) em staging e production. |
| Change Failure Rate | Adaptada: proporção de execuções de CI/CD que falham. |
| Mean Time to Restore | Parcialmente aplicável: tempo para corrigir um pipeline vermelho (por exemplo, a falha do CD resolvida trocando `pytest` por `python -m pytest`). |

### 10.3 Relatórios e Auditorias

O histórico de commits, Pull Requests (com aprovações), execuções do GitHub Actions e deploys por Environment compõe a trilha de auditoria. As configurações que não ficam em arquivo são comprovadas por prints em `docs/evidencias/`.

## 11. Marcos

- Código-fonte com as quatro operações básicas: concluído.
- Testes automatizados (pytest), incluindo divisão por zero: concluído.
- Pipeline de CI com lint e testes: concluído.

- Elaboração do PGCS v1.0: concluído.
- Publicação do plano de GCS e do template de PR no repositório, por Pull Request: concluído.
- Pipeline de CD com Environments staging e production: concluído.
- Proteção da `main` por ruleset e par de revisão: concluído.
- Evidências de conformidade versionadas em `docs/evidencias/`: concluído.
- Tags de versão (SemVer), Release automático e deploy real: proposto, não iniciado.

## 12. Treinamento e Recursos

Ferramentas: Git, conta no GitHub, Python 3.12, pytest e flake8 (via `requirements.txt`). Os conhecimentos de versionamento, testes automatizados, GitHub Actions, rulesets e Environments foram adquiridos ao longo da disciplina. O documento `docs/plano-gcs.md` e o repositório de referência da disciplina servem como material de consulta.

## 13. Controle de Software de Subcontratados, Fornecedores e Dependências de Terceiros

Não há subcontratados ou fornecedores. As bibliotecas de terceiros (pytest e flake8) são declaradas em `requirements.txt` e reinstaladas a cada execução do CI e do CD, garantindo consistência. Como melhoria futura, recomenda-se fixar versões exatas e habilitar o Dependabot para atualizações e verificação de vulnerabilidades (SCA).

## 14. Quadro-Resumo do Pipeline

| Etapa | Descrição | Gatilho |
|---|---|---|
| Pull Request | Branch `feature/`, `fix/` ou `docs/` com PR usando o template e título em Conventional Commits. | Abertura/edição do PR |
| CI (`build-and-test`) | Checkout, Python 3.12, dependências, flake8 e pytest. | Pull Request para a main |
| Revisão | 1 aprovação do revisor par. | Após o CI verde |
| Merge | Squash merge na main; push direto é recusado pelo ruleset. | Aprovação + CI verde |
| CD: build | pytest e geração do artefato `calculadora-dist`. | Push na main |

| Etapa | Descrição | Gatilho |
|---|---|---|
| CD: staging | Deploy automático (simulado). | Após o build |
| CD: production | Deploy (simulado) após aprovação de Elisa Maria Gomes. | Aprovação manual no Environment |

## 15. Evidências de Conformidade

Os prints abaixo ficam em `docs/evidencias/` no repositório.

| Requisito | Evidência | Arquivo |
|---|---|---|
| Estrutura do repositório e plano de GCS publicado | Pasta `docs/` com este plano e o `plano-gcs.md` | `docs/` |
| Repositório acessível para verificação | Repositório público | `docs/evidencias/colaboradores.png` (indica "Public repository") |
| Recusa do push direto na main | Mensagem do GitHub: não é possível commitar na `main` porque é uma branch protegida | `docs/evidencias/push-recusado.png` |
| Regra de proteção da main | Ruleset `proteger-main` ativo | `docs/evidencias/regra-main.png` |
| Par de revisão com acesso de escrita | @otavioroecker como Collaborator; PR aprovado por ele | `docs/evidencias/colaboradores.png` |
| Environments staging e production, este com a responsável como revisora | Configuração dos Environments | `docs/evidencias/environments.png` |
| Deploy em production aguardando e recebendo aprovação | Execução do CD com *waiting for review* e depois aprovada | `docs/evidencias/cd-aguardando-aprovacao.png` e `docs/evidencias/cd-aprovado.png` |

*Este documento foi atualizado para refletir o projeto Calculadora no estado atual: aplicação CLI com pipelines de CI e CD, ambientes staging e production controlados, branch main protegida e revisão por par. As etapas de tags, releases automáticos, deploy real e IaC permanecem como propostas de evolução.*